CONTENTS

Abstract **v**

1 Introduction **1**
1.1 DisCosTic . . . 1
1.2 ECM Model . . . 2

2 Integrate the ECM Model **3**
2.1 Establishing the Node Model . . . 3
2.2 Integration of Kerncraft . . . 4
2.3 Scaling of a Single-Core Performance Estimation . . . 4

3 Analysis **7**
3.1 Adaptability Analysis . . . 7
3.2 In-Depth Analysis . . . 7

4 Conclusion **11**
4.1 Outlook . . . 11
List of Figures . . . 13
List of Tables . . . 15
List of Listings . . . 17
Bibliography . . . 19

A Appendix **19**

1 INTRODUCTION

With the increased capabilities and performance of both supercomputers and clusters, more and more massively parallel programs are developed, often with the use of MPI and a parallization paradigm for the node-level, underneath, e.g. OpenMP. As a consequence of the increased portion of massively parallel programs, also comes an interest to study the behavior of such programs to ease playing with parameters and inquire about the effects that come with desyncronization, resynchronization, idle waves, computational waves and how the system and noise influences these. The problem with such a case study is often the amount of time and resources that need to be spend to run the program successfully. It is neither efficient nor cost effective, to run a program with new parameters to study the effects, when it takes days to complete.

To circumvent this problem we need a simulator that can simulate the excecution of a massively parallel code as accurately as possible. To assure the simulation is as accurate as possible, the simulator needs to be aware of the usual node-level bottlenecks that are present on modern systems, specifically the memory bandwidth bottleneck on ccNUMA domains. Furthermore, the dynamic between system, comunication and computation can result in deviations that have either a positive or negative influence on your runtime through noise, comunications time cariations and an inherent load imbalance. For example, under system noise fall phases of inactivity, like the intended inject of delays to both your execution and communication properties, called idle waves. These delays can happen for a multitude of reasons, one can be the lack of resources available or data dependency in your code. No current simulator takes a theoretical approach to both simulators, they either predict the node-level runtime or predict the communication time with the help of of measurements done on the node-level.

1.1 DisCosTic

One of these simulators is the Distributed Cost in Cluster (DisCosTic) framework. DisCosTic is a framework to predict the runtime for large-scale parallel applications, based on analytical first-principle performance models. It is based on graph generation, the analytic Roofline model and a simple latency-bandwidth model for computaion and communication respectively. Discostic works by first generating an abstract syntax tree (AST) to manage MPI message semantics and system topology. The topology and the semantics are provided by the user through a config, that contains the systems specifics, it includes the general information about the cluster, e.g. number of nodes, chips per chip, etc. It also sets the parameters for both the communication and computational model. For the structure of your code it is necessary to write an application model in the DisCosTic language, where it describes when and where to compute,send and receive messages. This whole information is then taken by DisCosTic to generate the AST.

The AST is then used as the basis for both the communication and computational model. DisCosTic models the compuation, communication and synchronization operations. The simple latency-bandwidth model uses the system specifics for latency and bandwidth provided in the config file, as well as the message size and number of messages send and received to estimate the amount of time needed for the communication and synchronization in one timestep.

The roofline model compares between the machine balance and intensity to determine if the code is memory or compute bound, depending on which part exceeds the other. If the code intensity surpasses the machine balence, the code is compute bound, as it will spend more time in computation than communication. The model is based on a Build and Bottleneck analysis and allows the consideration of the two principal performance boundaries computation and communication in isolation.

In its current state the DisCosTic framework uses measured runtimes of the specified kernel, when executed on a nodelevel, to extrapolate and estimate the execution time on a multinode cluster in combination of both computation and communication time. While the roofline model works with measured runtimes, it is not suited for an accurate estimation of chip-level performance, due to being too optimistic for real hardware and ignoring constraints, such as non-overlapping data transfers inside the memory hierarchy. As my task is to implement a chip-level performance model into the DisCosTic framework, I decided to use the Execution-Cache-Memory (ECM) performance model.

1.2 ECM Model

The ECM model was created to help with understanding the performance behavior of multi-core processors and their complex memory hierarchies. ECM predicts the single-core performance by estimation the number of CPU cycles that are required to execute a number of iterations of a loop on a single core. This prediction is comprised of the in-core execution time T_c and the transfer time T_d. While the time for a single data transfer is calculated at a cacheline granularity, the in-core ececution time must be broken down further depending on the LOAD and Store instructions that are retired in which cycle. This divides the execution time into the overlapping and non-overlapping parts, T_{OL} and T_{nOL} respectively and predicted by an analysis of the assembly code with all its dependencies. The execution time is then determined by the unit that takes the longest to retire its allocated instruction. Additionally, any additonal constraints must be taken into account, e.g. the Intel Sandy Bridge-EP microarchitecture can retire a maximum of four instructions per cycle. The data transfers are modeled under the assumption that transfer times are exclusively a function of bandwidth and not of latency, which is viable due to the very efficient modern prefetchers. The individual contributons for each cache level can then be determined by dividing the amount of data transfer with the available bandwidth, provided that the transfer between the L3-Cache and main memory receives special care, because the theoretical bandwidth and sustained bandwidth can differ significantly in some cases. As we are currently limiting ourselves to Intel CPU, all memory transfers are non-overlapping, therefore we can sum up the individual contributions to the total transfer time T_{data}. From there the ECM prediction T_{ECM} is the maximum between T_{OL} and $T_{nOL} + T_{data}$.

2 INTEGRATE THE ECM MODEL

2.1 Establishing the Node Model

Code 2.1 shows the basic variables that are needed to calculate the estimation of the node-level runtime. The machine struct has all the basic information about the node setup, like the number of cores, sockets and the frequency. The ECM struct contains the variables that represent the data gained by estimating the cyles per cacheline with the help of the ECM model. The variable T_{MECM} is the ECM prediction of a multi-core execution, while the rest represent the essential predictions of the single-core execution.

```
1  //===Machine===//
2  struct Machine{
3      DisCosTiC_Datatype n_cores_;
4      DisCosTiC_Datatype cores_per_numa_domain_;
5      DisCosTiC_Datatype cores_per_socket_;
6      DisCosTiC_Datatype sockets_;
7      DisCosTiC_Timetype f_core_;
8
9  };
10
11 //===ECM Model===//
12 struct ECM{
13     DisCosTiC_Timetype T_OL_;
14     DisCosTiC_Timetype T_nOL_;
15     DisCosTiC_Timetype T_L1L2_;
16     DisCosTiC_Timetype T_L2L3_;
17     DisCosTiC_Timetype T_L3Mem_;
18     DisCosTiC_Timetype T_ECM_;
19     DisCosTiC_Timetype T_MECM_;
20 };
```

Listing 2.1 – Necessary Variables for the Machine and ECM Model

The user can provide the data with the help of the four different options, as can be seen in A.1. The code 2.2 shows that the first option starts by using the config parser of the DisCosTiC framework to set the necessary variables in the node model, A.5, and then estimates the runtime with the help of equation 2.1 by converting the cycles per cacheline from the multi-core estimation into seconds, A.2.

$$t = \frac{T_{MECM}}{f} \tag{2.1}$$

```
if(opname.substr(0, n) == "LBL"){
      opname = opname.substr(n+1, opname.length());

    if(this->func  != opname)
      throw std::runtime_error("computation operation NOT YET IMPLEMENTED\n");

    UserInterface::ConfigParser NodeCFG("./nodelevel/configs/" + opname + ".cfg");
      NodeModel NM(&NodeCFG);

      estimation(NM, &perf_est, &runtime);
}
```

Listing 2.2 – The code uses to run the first option using a simple config file, like A.7.

2.2 Integration of Kerncraft

Normally, a user will not have the data provided by the ECM model at hand and would need to be calculated first. As the ECM Model is not only dependent on the code itself, but also on the machine on which the code is executed, most users will not have the knowledge to calculate the ECM model by hand. The second option in A.1 uses the integration of the external tool Kerncraft to calculate the particulars with the help of the necessary information provided by the source code, the machine, the number of cores and the number of iterations for the loop kernel. The external tool Kerncraft can automatically construct the ECM model for loop kernels. Kerncraft performs a code and data transfer analysis. If possible, Kerncraft also analyses the layer condition and has the fallback option of cache simulator. Code A.6 implements the execution of Kerncraft by starting an extra process, A.3.

The option string contains the name of the source code file, output name, the relative path to the machine file, the number of cores and the definition of the size of the loop kernel. This order is currently hardcoded and can not be changed or the string seperation will not work. With this information we set the all the arguments necessary to start the external of the Kerncraft script. The script changes the original output to append the data from the ECM module to the config file in the correct format. After the external process is done, the option does not differentiate from the first option. The third option hardcodes the source code into the routine provided by the user.

2.3 Scaling of a Single-Core Performance Estimation

The last option can be used in the cases the kernels are too complicated for Kerncraft, the single-core estimation was calculated by hand or a case study by changing the parameters around the ECM model. The ECM Model assumes that the performance on the caches scales linearly with the number of cores used. That means as long as the amount of data perfectly fits into the cache, the estimation is simply the number of cores times the single-core performance. The DisCosTiC framework simulates the execution of large scale parallel programs with a large amount of data. Therefore, the data size will probably not fit into the cache of a socket. Furthermore, as mentioned in the introdruction, the ccNUMA domain must be taken into account when sacling the performance for a multi-core processor.

In the case of data transfer from main memory to the L3-cache, the first step is to consider the utilization of the memory bus. Code 2.3 implements the calculations to set the exact number of cores the memory bus is saturated inside the NUMA domain.

```
util[0] = 0.0;
scaling_cores = m.cores_per_socket_ * m.sockets_ + 17;
for(DisCosTiC_Datatype c = 1; c < m.cores_per_numa_domain_ + 1; ++c){
  if(c * ecm.T_L3Mem_ > (ecm.T_ECM_ + util[c-1] * (c-1) * ecm.T_L3Mem_ * 0.5)){
    util[c] = 1.0;
    scaling_cores = std::min(scaling_cores, c);
  }else{
    util[c] = (c * ecm.T_L3Mem_) / (ecm.T_ECM_ + util[c-1] * (c-1) * ↘
        (ecm.T_L3Mem_ * 0.5));
  }
}
```

Listing 2.3 – Calculation of the memory bus utilization.

After that, the code snippet shown in 2.4, starts predicting the performance, dependent on the number of cores used and the saturation of the memory bus, this differs from kernel to kernel. The variable `scaling_cores` represents the exact number of cores, at which the saturation level is reached. This variable determines the performance inside the numa domain. Next, a check is done to see if the numbers of cores used is inside the NUMA domain, or if we have exceeded the number of cores per NUMA domain. As a failsafe, a check to not exceed the number of cores on the socket a third option is implemented. The last step includes the simple call to the estimation method, as shown in A.2.

```
for(DisCosTiC_Datatype c = 1; c < m.cores_per_socket_ + 1; ++c){

  DisCosTiC_Timetype numa = 0.0;
  if(c <= scaling_cores){
    numa = ecm.T_ECM_ / (ecm.T_ECM_ / ecm.T_L3Mem_ );
  }else{
    numa = ecm.T_L3Mem_;
  }
  scaling_numa[c-1] = numa;

  if(0 < c && c <= m.cores_per_numa_domain_){
    scaling_performance[c-1] = (numa/util[c]);
  }else if(m.cores_per_numa_domain_ < c && c <= m.cores_per_socket_ * ↘
      m.sockets_){
    scaling_performance[c-1] = (numa/util[m.cores_per_numa_domain_]) * ↘
        (static_cast<DisCosTiC_Timetype>(m.cores_per_numa_domain_)/ ↘
        static_cast<DisCosTiC_Timetype>(c));
  }else{
    std::cerr << "The number of cores must be greater than zero and only ↘
        upto the max number available as defined by cores per socket and ↘
        sockets in the machine file." << std::endl;
    exit(EXIT_FAILURE);
  }
}
```

Listing 2.4 – The code snippet to predict the performance for the number of cores per socket.

3 ANALYSIS

To prove the benefit of the integration of a theoretical model, like the ECM model, into the DisCosTiC framework, different kernels need to be tested for both adaptability for the different options provided by the interface. This thesis limits itself to the Intel Broadwell-EP E5-2967, with a frequency of 2.3GHz. In the appendix A.8, the bodies are displayed. These kernels were chosen for there simplicity, diversity and that these are often used in a variety of large scale parallel programs.

3.1 Adaptability Analysis

In this section, all kernels are tested for their adaptability between the two different Intel CPUs. The kernels were executed with 18 cores on a single Broadwell-EP E5-2967 CPU.
Table 3.1 shows us the predicted runtime provided by the DisCosTiC framework, when executing the same kernel with the different options. It also provides the evidence that the inclusion of the Kerncraft framework and a theoretical option do not change the overall outcome of the prediction.

Kernel	Option 1: LBL	Option 2: FILE	Option 3: SRC	Option 4: COMP
COPY	43.6094	43.6097	43.6102	43.6092
SCALE	41.5141	41.5146	41.5154	41.5149
SUM	69.1859	69.1864	69.1868	69.1863
STREAM-TRIAD	51.7785	51.7778	51.7777	51.7779
DAXPY	57.5485	57.5488	57.5492	57.5482
Schoenauer-TRIAD	51.0537	51.0542	51.0546	51.05428
SCALAR-PRODUCT	58.6599	58.6503	58.6591	58.65
vectorSUM	49.162	49.1617	49.1611	49.1615
2D-5PT-STENCIL	51.4871	51.4874	51.4866	51.4882

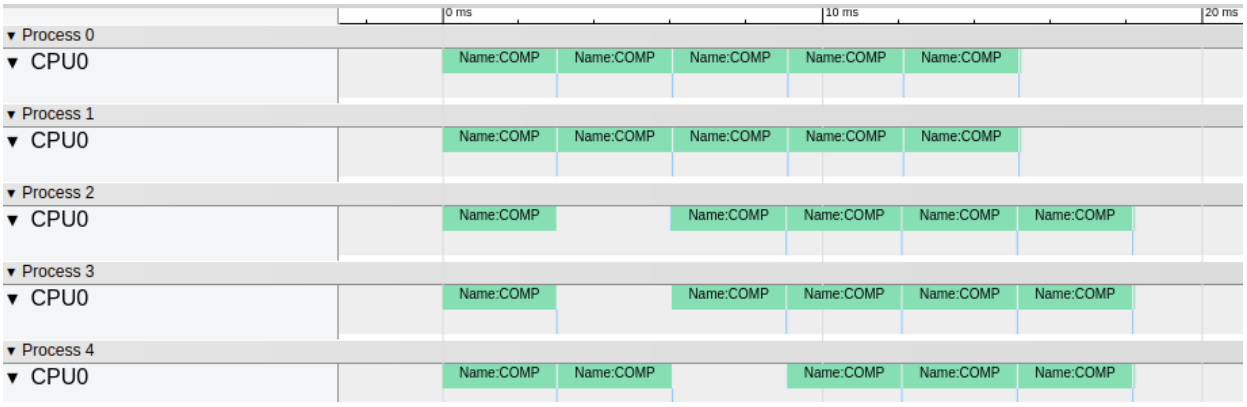
Table 3.1 – Runtime in [s] of the different Kernels on an Intel Broadwell-EP E5-2967 with the array size of 100000.

3.2 In-Depth Analysis

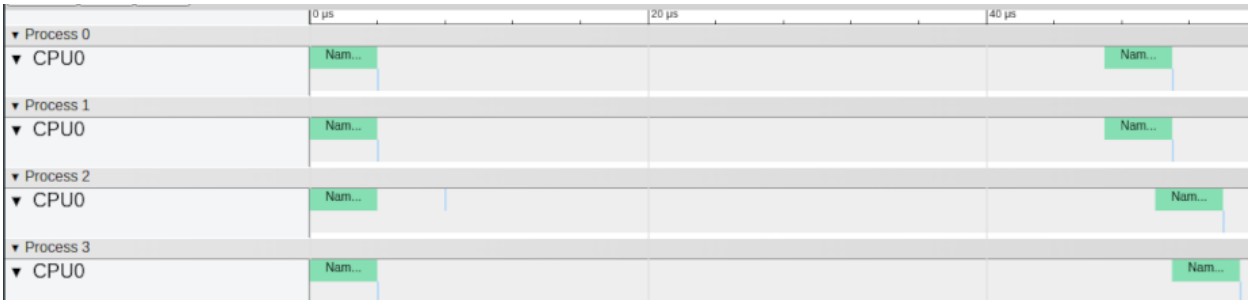
The in-depth analysis is concentrated on the Stream-Triad kernel and the 2d-5pt Stencil. The stream-triad kernel is very often used for performance benchmarks and the stencil is depedendent on the data provided by the neighbors.

3.2 In-Depth Analysis

The figure 3.1 clearly indicates the difference that comes from the ECM model. In subfigure 3.1a, we see the classical pipeline structure of the code with a bit of ideal time. On the other hand, the subfigure 3.1b gives us evidence that the ideal time between computations is not only dependent on the system noise, data transfer between main memory and cache, etc. This means, that for the stream triad kernel, the integration of ECM model results in a more accurate and reliable prediction of the runtime as a whole.



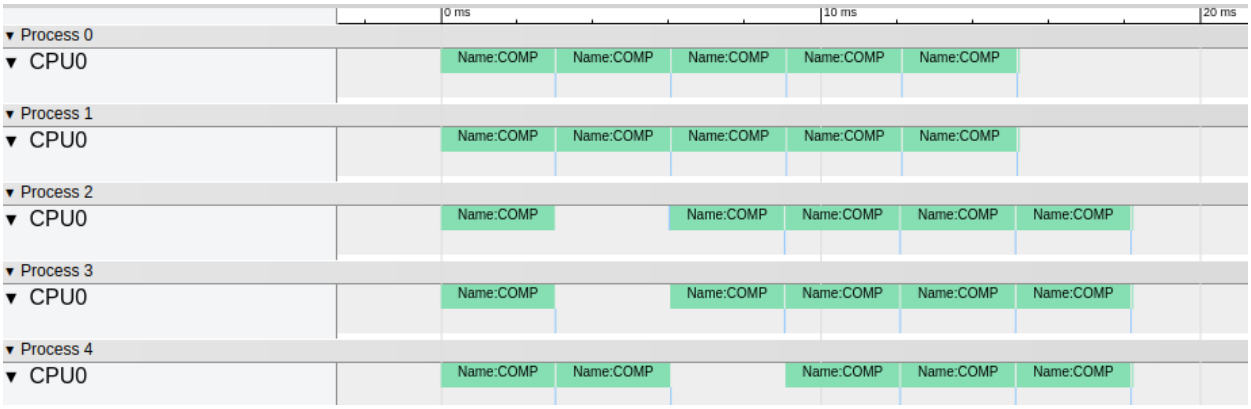
(a) Trace of the Stream-Triad kernel without ECM Model



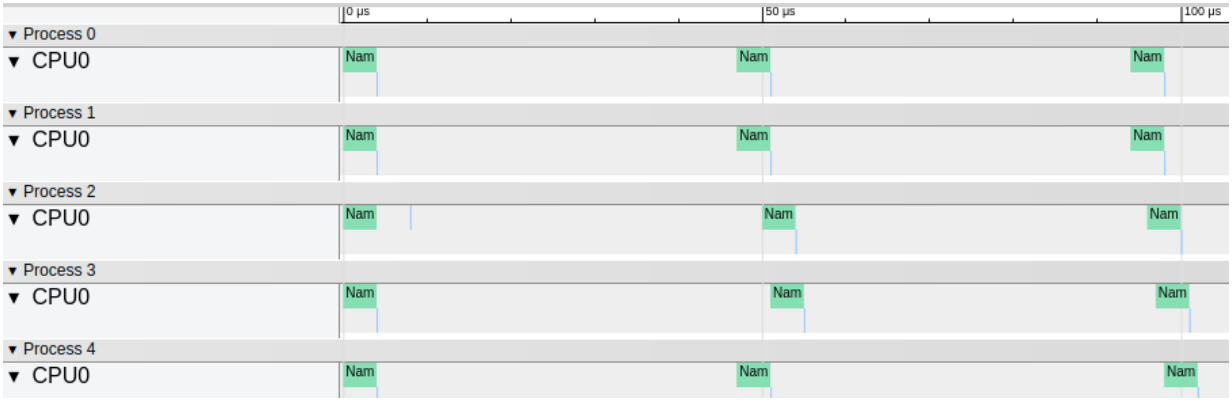
(b) Trace of the Stream-Triad kernel with ECM Model

Figure 3.1 – The traces of the Stream Triad kernel executed with the different versions of DisCosTiC.

Figure 3.2 corresponds with the results seen above. Interesting is that subfigure 3.2a shows us the same exact picture to the trace shown for a stream triad kernel. This proves that without the integration of an analytic performance model, the end trace is completely dependent on the variance in the communication and the system network. As both kernels were executed on a single CPU, there was no requirement for any MPI communication, the predicted runtime will be the same, when the node-level runtime is the same. This is the drawback, due to the missing variance in data dependency and transfer time in the memory hierarchy.
The subfigure 3.2b further provides us with evidence, that the inclusion of the ECM model will result in more accurate predictions, as the processess can even shift their computations slot between timesteps. On timestep two process four was even with process 0 and 1, but on the third timestep it starts later than process 3. There is no evidence of this delays that can be seen in the traces created by the original DisCosTiC framework.



(a) Trace of the 2d-5pt stencil without ECM Model



(b) Trace of the 2d-5pt stencil with ECM Model

Figure 3.2 – The traces of the 2d-5pt stencil executed with the different versions of DisCosTiC.

4 CONCLUSION

To get a more accurate prediction of the runtime for a parallel program the inclusion of an analytical chip-level performance model, like the ECM model, was necessary. With the help of a such a model, the variance of system, network structure and computation kernel is taken into account when predicting the runtime of large scale parallel programs.
The adaptability analysis shows us that the inclusion has no drawbacks when it comes to the overall user friendliness, as the interface is not that much different, even given you the additional options in how to generate or provide the input necessary for the framework as a whole. While a more in-depth analysis shows us the benefits of a simulator with an analytical approach to the chip-level, as the results gave us the evidence, that the measured runtimes, will only show real variance when an active MPI communication exists, otherwise the difference would only be in how long the computational phases for each process lasts. It also presents us the evidence, how a delay can effect each process individually.

4.1 Outlook

The results of this thesis can be extended in a few different direction. The first direction would be an easier and more direct interface the Kerncraft framework, without the need of starting an external process. This would help in the handling of both the necessary arguments to start Kerncraft and its output.
The second direction would be case study, with inclusion of a greater variety in Intel processes, in how close the DisCosTiC framework comes to an accurate prediction when comparing the simulation to the execution of the real large scale parallel application.
Another direction is the extension of the ECM model to also incorporate processers that allow for overlapping memory transfers between the caches. This would bring a few different challenges to the current model, as the scaling method would need to be adjusted.

APPENDIX



```
1 DisCosTiC->Exec("LBL:STREAM_TRIAD",
2         DisCosTiC::Event depending_operations);
3 DisCosTiC->Exec("COMP:TOL=2.0||TnOL=1.0|TL1L2=3.0|TL2L3=6.0|TL3Mem=14.2",
4         DisCosTiC::Event depending_operations);
5 DisCosTiC->Exec("FILE:copy.c//BREAK:COPY//./BroadwellEP_E5-2697v4_CoD.yml//18//
6         -D N 100000", DisCosTiC::Event depending_operations);
7 DisCosTiC->Exec("SRC:double s, a[N],b[N],c[N];\n\nfor(int i=0; i<N; i++)
8         \n\ta[i]=b[i]+s*c[i];\n//BREAK:DAXPY//./BroadwellEP_E5-2697v4_CoD.yml
9         //18//-D N 100000", DisCosTiC::Event depending_operations);
```

Listing A.1 – The different input options in how the user can interact with the node-level perormance analysis.

```
void estimation(NodeModel& NM, DisCosTiC_Timetype* perf_est, ↘
    DisCosTiC_Timetype* runtime){
  Machine machine = NM.getMachine();
  ECM ecm = NM.getECM();
  DisCosTiC_Datatype flops = NM.getFlops();
  *perf_est = (flops / ecm.T_MECM_) * (machine.f_core_ * 1000); //This gives ↘
      me the MFlop/s
  *runtime = ecm.T_MECM_ / (machine.f_core_ * 1e9);
}
```

Listing A.2 – Code for the runtime estimation.

```
void executeKerncraft(char* argV[]){

  pid_t pid = fork(); /* Create a child process */

  switch (pid) {
    case -1: /* Error */
      exit(1);
    case 0: /* Child process */
      execvp(argV[0], argV); /* Execute the program */
      exit(1);
    default: /* Parent process */
      int status = 0;

      waitpid(pid, &status, 0); /* Wait for the process to complete */

      return;
  }
}
```

Listing A.3 – Code for the execution of the kerncraft script.

```
void scaling(NodeModel& NM, DisCosTiC_Timetype* scaling_performance, ↘
    DisCosTiC_Timetype* scaling_numa){
  ECM ecm = NM.getECM();
  Machine m = NM.getMachine();
  if(ecm.T_L3Mem_ != 0.0){
    DisCosTiC_Timetype util[m.cores_per_numa_domain_+1];
    DisCosTiC_Datatype scaling_cores;

    //Calculation of Utilization
    util[0] = 0.0;
    scaling_cores = m.cores_per_socket_ * m.sockets_ + 17;
    for(DisCosTiC_Datatype c = 1; c < m.cores_per_numa_domain_ + 1; ++c){
      if(c * ecm.T_L3Mem_ > (ecm.T_ECM_ + util[c-1] * (c-1) * ecm.T_L3Mem_ * ↘
          0.5)){
        util[c] = 1.0;
        scaling_cores = std::min(scaling_cores, c);
      }else{
        util[c] = (c * ecm.T_L3Mem_) / (ecm.T_ECM_ + util[c-1] * (c-1) * ↘
            (ecm.T_L3Mem_ * 0.5));
      }

    for(DisCosTiC_Datatype c = 1; c < m.cores_per_socket_ + 1; ++c){

      DisCosTiC_Timetype numa = 0.0;
      if(c <= scaling_cores){
        numa = ecm.T_ECM_ / (ecm.T_ECM_ / ecm.T_L3Mem_ );
      }else{
        numa = ecm.T_L3Mem_;
      }
      scaling_numa[c-1] = numa;

      if(0 < c && c <= m.cores_per_numa_domain_){
        scaling_performance[c-1] = (numa/util[c]);
      }else if(m.cores_per_numa_domain_ < c && c <= m.cores_per_socket_ * ↘
          m.sockets_){
        scaling_performance[c-1] = (numa/util[m.cores_per_numa_domain_]) * ↘
            (static_cast<DisCosTiC_Timetype>(m.cores_per_numa_domain_)/ ↘
            static_cast<DisCosTiC_Timetype>(c));
      }else{
        std::cerr << "The number of cores must be greater than zero and only ↘
            upto the max number available as defined by cores per socket and ↘
            sockets in the machine file." << std::endl;
        exit(EXIT_FAILURE);
      }
    }
  }else{
    for(DisCosTiC_Datatype c = 1; c < m.cores_per_socket_ + 1; ++c){
      scaling_performance[c-1] = ecm.T_ECM_/static_cast<DisCosTiC_Timetype>(c);
      scaling_numa[c-1] = ↘
          ecm.T_ECM_/static_cast<DisCosTiC_Timetype>(m.cores_per_numa_domain_);
    }
  }

}
```

Listing A.4 – Scaling method to scale the prediction from a single-core to multi-core.

```
class NodeModel {
  private:
    Machine machine_;
    ECM ecm_;
    DisCosTiC_Datatype flops_;
    std::string filename_;
  public:
//===Constructor===//
    NodeModel() = default;
    NodeModel(Machine& machine, ECM& ecm, DisCosTiC_Datatype flops, ↘
        std::string& filename) : machine_(machine), ecm_(ecm), ↘
        flops_(flops), filename_(filename) {}
    NodeModel(UserInterface::ConfigParser *CFG_args) {
      machine_.n_cores_ = ↘
          CFG_args->getValue<DisCosTiC_Datatype>("n_cores");
      machine_.cores_per_numa_domain_ = ↘
          CFG_args->getValue<DisCosTiC_Datatype>("cores_per_numa_domain");
      machine_.cores_per_socket_ = ↘
          CFG_args->getValue<DisCosTiC_Datatype>("cores_per_socket");
      machine_.sockets_ = CFG_args->getValue<DisCosTiC_Datatype>("sockets");
      machine_.f_core_ = CFG_args->getValue<DisCosTiC_Timetype>("f_core");

      ecm_.T_OL_ = CFG_args->getValue<DisCosTiC_Timetype>("T_OL");
      ecm_.T_nOL_ = CFG_args->getValue<DisCosTiC_Timetype>("T_nOL");
      ecm_.T_L1L2_ = CFG_args->getValue<DisCosTiC_Timetype>("T_L1L2");
      ecm_.T_L2L3_= CFG_args->getValue<DisCosTiC_Timetype>("T_L2L3");
      ecm_.T_L3Mem_ = CFG_args->getValue<DisCosTiC_Timetype>("T_L3Mem");
      ecm_.T_ECM_ = CFG_args->getValue<DisCosTiC_Timetype>("T_ECM");
      ecm_.T_MECM_ = CFG_args->getValue<DisCosTiC_Timetype>("T_MECM");
      filename_ = CFG_args->getValue<std::string>("filename");
      flops_ = CFG_args->getValue<DisCosTiC_Datatype>("FLOPs_per_iteration");
    }
    NodeModel(UserInterface::ConfigParser *CFG_args, DisCosTiC_Timetype ↘
        T_OL, DisCosTiC_Timetype T_nOL, DisCosTiC_Timetype T_L1L2, ↘
        DisCosTiC_Timetype T_L2L3, DisCosTiC_Timetype T_L3Mem, ↘
        DisCosTiC_Timetype T_ECM) {
      machine_.n_cores_ = ↘
          CFG_args->getValue<DisCosTiC_Datatype>("n_cores");
      machine_.cores_per_numa_domain_ = ↘
          CFG_args->getValue<DisCosTiC_Datatype>("cores_per_numa_domain");
      machine_.cores_per_socket_ = ↘
          CFG_args->getValue<DisCosTiC_Datatype>("cores_per_socket");
      machine_.sockets_ = CFG_args->getValue<DisCosTiC_Datatype>("sockets");
      machine_.f_core_ = CFG_args->getValue<DisCosTiC_Timetype>("f_core");

      ecm_.T_OL_ = T_OL;
      ecm_.T_nOL_ = T_nOL;
      ecm_.T_L1L2_= T_L1L2;
      ecm_.T_L2L3_= T_L2L3;
      ecm_.T_L3Mem_= T_L3Mem;
      ecm_.T_ECM_ = T_ECM;
      ecm_.T_MECM_ = 0.0;
      filename_ = CFG_args->getValue<std::string>("filename");
      flops_ = CFG_args->getValue<DisCosTiC_Datatype>("FLOPs_per_iteration");
    }
};
```

Listing A.5 – Code for the NodeModel class

```
else if(opname.substr(0, n) == "FILE"){
  opname.erase(0,n+1);
  std::string::size_type opos = opname.find(":");
  std::string::size_type machinePos = opname.find("//", opos);
  std::string output = "nodelevel/configs/" + opname.substr(opos+1, machinePos-opos-1) + ".cfg";
  if(!kerncraftExecuted){
    kerncraftExecuted = true;
    std::string::size_type pos = opname.find("//BREAK:");
    std::string source = "nodelevel/kernels/" + opname.substr(0,pos);
    std::string::size_type opos = opname.find(":");
    std::string::size_type machinePos = opname.find("//", opos);
    std::string::size_type corePos = opname.find("//", machinePos+2);
    std::string::size_type definePos = opname.find("//", corePos+2);
    std::string script("./scripts/diskern.py");
    std::string model("-pECM");
    std::string outputPath = "-o" + output;
    std::string machine = "-m" + opname.substr(machinePos+2, corePos-machinePos-2);
    std::string cores = "-c" + opname.substr(corePos+2, definePos-corePos-2);
    std::string defines = opname.substr(definePos+2, opname.length());
    char scriptPath[script.length()+1];
    char modelType[model.length()+1];
    char outputFile[outputPath.length()+1];
    char machineFile[machine.length()+1];
    char coreNumber[cores.length()+1];
    char keyValues[defines.length()+1];
    char sourceFile[source.length()+1];

    std::strcpy(scriptPath, script.c_str());
    std::strcpy(modelType, model.c_str());
    std::strcpy(outputFile, outputPath.c_str());
    std::strcpy(machineFile, machine.c_str());
    std::strcpy(coreNumber, cores.c_str());
    std::strcpy(keyValues, defines.c_str());
    std::strcpy(sourceFile, source.c_str());

    std::vector<char*> argVek {scriptPath, modelType, machineFile, coreNumber, outputFile};
    char* token = std::strtok(keyValues, " ");
    while (token != NULL) {
      argVek.push_back(token);
      token = std::strtok(NULL, " ");
    }
    argVek.push_back(sourceFile);
    char *argV[argVek.size()+1];
    for(std::size_t i = 0; i < argVek.size(); ++i){
      argV[i] = argVek[i];
    }
    argV[argVek.size()] = NULL;
    executeKerncraft(argV);
  }
  UserInterface::ConfigParser NodeCFG(output);
  NodeModel NM(&NodeCFG);
  estimation(NM, &perf_est, &runtime);
}
```

Listing A.6 – Execution of an external source code with the integration of Kerncraft.

```
#///    \file
#///    STREAM_TRIAD_CFG
#///

filename = stream_triad.txt

#Machine
n_cores = 18
f_core = 2.3
cores_per_numa_domain = 9
cores_per_socket = 18
sockets = 2

#Workload
FLOPs_per_iteration = 8

#ECM
T_OL = 3.3
T_nOL = 2.0
T_L1L2 = 4.0
T_L2L3 = 8.0
T_L3Mem = 19.0
T_ECM = 33.0
T_MECM = 9.50
```

Listing A.7 – Example config file for a Stream-Triad kernel.

```
1 COPY: a[i] = b[i]
2 SCALE: a[i] = s * b[i]
3 SUM: a[i] = b[i] + c[i]
4 STREAM-TRIAD: a[i] = b[i] + s * c[i] (temporal vs. non-temporal store)
5 DAXPY: a[i] = a[i] + s * b[i]
6 Schoenauer-TRIAD: a[i] = b[i] + c[i] * d[i]
7 scalarProduct: s += a[i]*b[i]
8 vectorSUM: s += a[i]
9 2D-5PT-STENCIL: b[j][i] = (a[j][i-1] + a[j][i+1] + a[j-1][i] + a[j+1][i]) * s
```

Listing A.8 – The different kernels that were tested and analysed